

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

A Prometheus exporter for Modbus connected devices - enabling continuous energy monitoring

Progetto di ingegneria informatica

Author: **Burroni Blu**

Advisor: Prof. Barenghi Alessandro

Academic Year: 2025-2026

Abstract

Il progetto di Ingegneria Informatica è stato rivolto sia a prendere conoscenza e dimestichezza con i sistemi di monitoraggio d'uso attuale nell'ambito dell'amministrazione di sistemi informatici, sia a confrontarsi con lo sviluppo di codice in un caso pratico inerente allo stesso ambito.

Questo quindi si è tradotto prima nella costruzione di un'infrastruttura di monitoring di prova e, poi, nella scrittura di un "collector-exporter" in Python, uno script che permette di integrare nel calcolo della Power Usage Effectiveness metriche lette da due multimetri tramite protocollo Modbus.

Parole chiave: Sistemi di monitoraggio, Grafana, Prometheus, PUE, Modbus.

Contents

Abstract **i**

Contents **iii**

1 Infrastruttura di monitoring **1**
1.1 Prometheus e Grafana . . . 1
1.2 Infrastruttura di test . . . 1
1.2.1 Docker . . . 2
1.2.2 Raccolta metriche . . . 2
1.3 Power Usage Effectiveness . . . 2
1.3.1 Calcolo del PUE con Grafana . . . 3

2 Modbus collector/exporter **5**
2.0.1 Modbus . . . 5
2.0.2 OpenModbusSpecs . . . 6
2.1 Struttura e best practices . . . 6
2.2 Sviluppo . . . 7
2.2.1 Testing . . . 7
2.2.2 Risultati . . . 8

3 Conclusione e riflessioni **9**
3.1 Un futuro Modbus Exporter . . . 9

Bibliography **11**

List of Figures **13**

1 | Infrastruttura di monitoring

IBM definisce il monitoraggio di infrastruttura IT come il processo di monitoraggio, analisi e gestione delle prestazioni, della disponibilità e dello stato di salute dei componenti di backend dello stack tecnologico di un'azienda[5].

La prima parte di questo progetto è stata incentrato sullo studio di due sistemi ampiamente utilizzati in questo ambito.

1.1. Prometheus e Grafana

Da uno studio della documentazione ufficiale è possibile comprendere e riassumere l'architettura e il funzionamento di Prometheus e Grafana come uno "stack" (i.e. una combinazione di programmi) di monitoring che permette di raccogliere, mantenere e visualizzare una grande varietà di metriche.

Prometheus[12] è un sistema open source di monitoraggio e alerting nato originariamente all'interno di SoundCloud[14] che permette di raccogliere e immagazzinare metriche come *time series*, ossia come informazioni collegate a una data (*timestamp*). Queste informazioni sono quindi mantenute in un TSDB (Time Series Database) che è possibile interrogare tramite il linguaggio PromQL (Prometheus Query Language) per selezionare e aggregare metriche[13] di vario tipo.

Grafana[3] (oltre che un ecosistema) è uno strumento che permette di aggregare metriche da varie fonti per poi poterle esplorare, interrogare e visualizzare. Ha un forte ecosistema e anche un sofisticato sistema di alerting.[4]

1.2. Infrastruttura di test

Per meglio apprendere questi contenuti, la fase successiva è stata quella di ricreare un ambiente in cui simulare lo stack di monitoring distribuito utilizzando Docker.

1.2.1. Docker

Docker[1] è una piattaforma per sviluppare, distribuire ed eseguire applicazioni che permette di separare quest'ultime dall'infrastruttura. In generale la containerizzazione (realizzata anche tramite soluzioni alternative a Docker, e.g. Podman, tutte basate sulla Open Container Initiative[6]) risolve il problema di applicazioni che funzionano su un'infrastruttura e non su un'altra virtualizzando una parte del sistema operativo (al contrario delle classiche Virtual Machines, riutilizzano il Kernel per minore overhead).

Docker Compose[2] permette di sfruttare tutti i vantaggi di Docker per applicazioni multi container definendo in un unico file yaml tutte le informazioni necessarie a gestire un gruppo di container.

1.2.2. Raccolta metriche

Definito un 'docker compose' per i due servizi Prometheus e Grafana e configurate le impostazioni iniziali (e.g. utente grafana) mancano solamente delle metriche reali su cui poter lavorare.

Per raccogliere metriche si è utilizzato un Prometheus Node Exporter: un Exporter[10] è un programma che espone metriche, in questo caso raccolte dal sistema operativo (e.g. utilizzo di CPU, occupazione di memoria...), sull'endpoint HTTP poi interrogato da Prometheus.

Una volta quindi che l'Exporter è in esecuzione sul nodo da monitorare, configurando lo *scraping* (aggiungendo un target nel file di configurazione 'prometheus.yml'), l'istanza Prometheus salverà le metriche sul TSDB e queste saranno quindi visualizzabili da Prometheus stesso.

Possiamo poi collegare Grafana a Prometheus dalla sezione "*Connessioni*" e poi "*Data sources*" così da poter creare una nuova *dashboard* da popolare con pannelli in cui specifichiamo le richieste da fare al database di Prometheus.

In figura (1.1) la semplice dashboard sviluppata.

1.3. Power Usage Effectiveness

La Power Usage Effectiveness (1.1) è un indicatore di efficienza energetica di un centro di elaborazione dati che mette in rapporto quanta energia è usata dalla componente

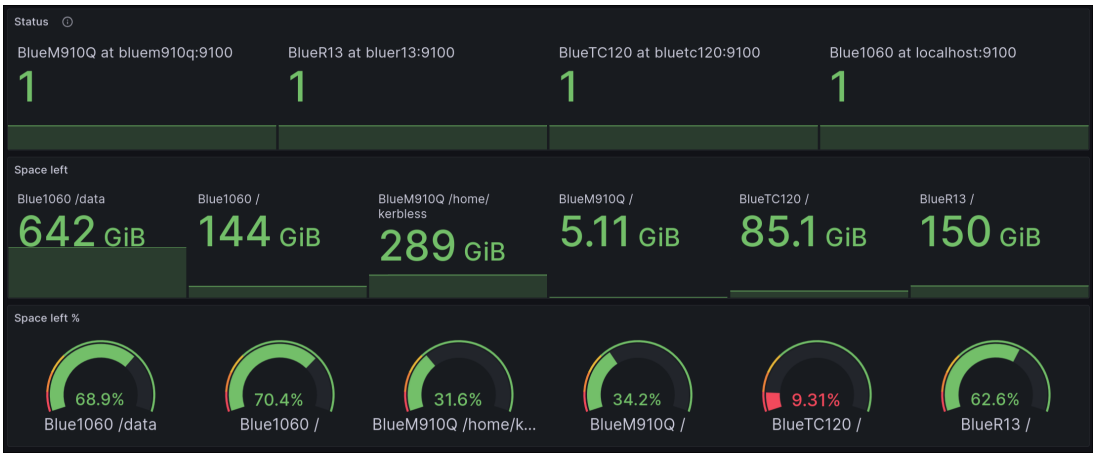


Figure 1.1: Semplice dashboard Grafana

computazionale a fronte di altri costi energetici (e.g. raffreddamento).

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Equipment Energy}} = 1 + \frac{\text{Non IT Facility Energy}}{\text{IT Equipment Energy}} \tag{1.1}$$

1.3.1. Calcolo del PUE con Grafana

Utilizzando l'infrastruttura di prova è stato possibile importare all'interno dell'istanza Prometheus alcune informazioni storiche sulla sala macchine del Dipartimento di Elettronica, Informazione e Bioingegneria. Dopo un po' di ricerca e studio delle metriche è stato possibile creare una nuova dashboard di Grafana per calcolare il PUE con le informazioni a disposizione (Figura 1.2). I risultati sono verosimili ma ottimistici: non tengono conto di alcune metriche il cui ottenimento è proprio l'obiettivo della seconda parte di questo progetto (cfr. capitolo 2).



Figure 1.2: Dashboard Grafana con un primo calcolo di PUE

2 | Modbus collector/exporter

In questa parte si è posto come obiettivo quello di ottenere delle metriche da due multimetri Schneider Electric PM3250 utilizzati nella sala macchine del DEIB che comunicano tramite Modbus RTU in modo da effettuare un calcolo meno approssimato del PUE.

2.0.1. Modbus

Modbus® è un protocollo di comunicazione, inizialmente sviluppato da Modicon nel 1979, che si posiziona al livello 7 del modello OSI[9] ed è tutt'ora standard de facto in molti ambienti industriali e di manifattura.[8]

Il protocollo è di tipo client-server e request-reply con un suo formato dati che lavora in modo indipendente dal livello di rete utilizzato per l'implementazione.

Specifica Modbus

[9]

Un messaggio è composto da un'Application Data Unit (ADU) dipendente dal livello di rete che può essere seriale o appoggiato su TCP/IP ed Ethernet (utilizzando la porta riservata 502). L'ADU incapsula una Protocol Data Unit (PDU) composta da:

- un campo function code, di 1 byte, che identifica l'azione richiesta. I codici funzione sono divisi in pubblici, definiti dall'utente o riservati per alcune aziende;
- un campo dati di lunghezza dipendente dal codice funzione (contiene e.g. indirizzi di variabili o sotto-codici funzione).

Il client che invia la PDU può ricevere una risposta normale, con il codice funzione che coincide con quello della richiesta e il campo dati con le informazioni richieste, o un'eccezione con il bit più significativo a 1 e il campo dati con un codice di eccezione.

Il modello dati di Modbus (Tabella 2.1) è suddiviso in quattro tabelle indirizzabili tramite indici interi senza segno.

Si **noti bene** che il riferimento nella PDU (da **0** a 65535) diverge dalla numerazione che si trova nel modello dati (da 1 a n): un dato numerato x nel modello Modbus viene indirizzato nella PDU come $x - 1$.

Inoltre le operazioni di lettura o scrittura possono interessare più elementi consecutivi, fino a un limite che dipende dal codice funzione.

Tabella	Tipo di oggetto	Accesso	Note
Discrete Input	singolo bit	solo lettura	tipicamente fornito da un sistema di I/O
Coils	singolo bit	lettura/scrittura	alterabile da un programma applicativo
Input Register	word a 16 bit	solo lettura	tipicamente fornita da un sistema di I/O
Holding Register	word a 16 bit	lettura/scrittura	alterabile da un programma applicativo

Table 2.1: Modello dati di Modbus[9].

2.0.2. OpenModbusSpecs

OpenModbusSpecs è un progetto open source in fase di sviluppo che tenta di risolvere il problema di dover gestire manualmente le specifiche dei dispositivi che utilizzano Modbus tramite una specifica utilizzabile meccanicamente creata con un JSON Schema.[7]

Poiché il passaggio di fornire uno standard per la definizione delle informazioni sul dispositivo da interrogare tramite Modbus era necessario, si è deciso di utilizzare lo schema fornito da questo progetto per evitare una specifica troppo informale e allo stesso tempo di dover costruirne una formale da zero.

Il risultato è un "device profile" (un file yaml) completamente aderente alla specifica del il progetto ma incompleto rispetto al manuale del dispositivo per questioni discusse in conclusioni.

2.1. Struttura e best practices

Uno studio iniziale delle tecnologie è stato necessario per sviluppare codice più possibile coerente con le *best practices*.

Per quanto riguarda Prometheus vi è una libreria Python ufficiale che permette di esporre i dati secondo il formato e le modalità consigliate dalla documentazione (e.g. inviare il valore di default 0 in caso di mancanza di dati etc.). Le uniche attenzioni, comunque rilevanti, sono state di attenersi alle linee guida per la nomina delle metriche Prometheus.[11]

Per la raccolta dati tramite Modbus invece si è deciso di utilizzare la libreria open source

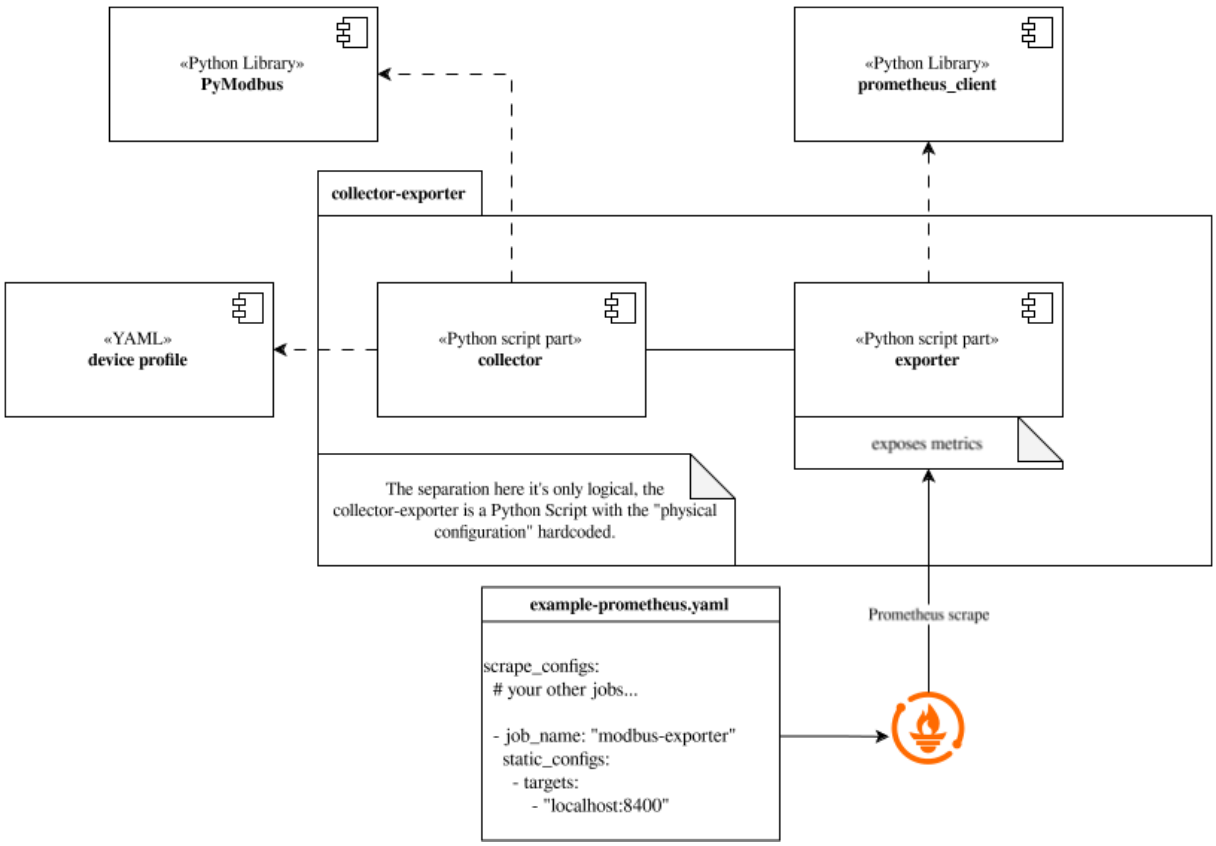


Figure 2.1: Struttura collector-exporter.py

PyModbus[15].

Lo script sviluppato è quindi strutturato come in figura (2.1).

2.2. Sviluppo

Lo sviluppo non è stato ovviamente lineare ma una volta concepito lo schema sopra esposto e aver preso dimestichezza con utilizzo delle librerie (purtroppo la documentazione di PyModbus non è eccellente) l'impegno è stato principalmente necessario per l'integrazione dei componenti e la scrittura del "*device profile*" (necessariamente manuale per garantire la correttezza del risultato). Del tempo è poi stato dedicato a delle considerazioni sulle prestazioni di lettura (lo script include una piccola parte di logica che permette di leggere registri in gruppi). Da qui i compromessi discussi in conclusione.

2.2.1. Testing

Ottenuta una prima bozza e poi successivamente a ogni modifica consistente il programma è stato testato ricreando un ambiente che permettesse la verifica delle funzionalità (nei

limiti della simulazione): utilizzando il programma "Socat" e la modalità "server" di PyModbus è possibile testare la raccolta di metriche da un dispositivo simulato e verificare la funzionalità dell'export tramite un'istanza Prometheus. Sono poi state annotate le modifiche che si prevedono necessarie per il funzionamento nella pratica (cfr. con le conclusioni).

2.2.2. Risultati

Il codice è ora utilizzato sul campo e raccoglie correttamente metriche dai multimetri. Una volta ottenute una quantità sufficiente di metriche sarà possibile ricalcolare il PUE che tuttavia non sarà facilmente confrontabile con i precedenti risultati per via di una ristrutturazione della sala macchine.

Il codice è consultabile dal repository pubblico: https://github.com/kerbless/modbus-collector-prometheus-exporter.

3 | Conclusione e riflessioni

Per via delle scelte fatte e per una questione di tempo, vi sono alcuni compromessi nel codice come sviluppato attualmente. I due principali:

- Non è possibile, senza modifiche al codice stesso, leggere più dispositivi di tipo diverso (con diverso "device profile").
- Non è possibile gestire metriche che non siano di tipo float32.

Questi limiti nascono però da una considerazione molto pratica: l'obiettivo in questo caso era di creare uno script che fosse ad-hoc. L'aspetto di generalizzazione e riutilizzo del codice è stato assolutamente considerato ma non applicato in pratica. Si raccoglie qui di seguito come si ritiene opportuno procedere con lo sviluppo di questo progetto per creare software applicabile su più larga scala.

3.1. Un futuro Modbus Exporter

Sviluppando nuovamente il progetto adottando il paradigma della programmazione a oggetti e quello dei design pattern (e.g. adapter, facade...) è possibile rendere il lavoro più riutilizzabile dal punto di vista del codice. Affiancando l'applicazione con un file di configurazione che tenga conto degli aspetti che potrebbero interessare l'utente finale (configurazione dei dispositivi e delle singole metriche da leggere, indirizzi Modbus assegnati ai dispositivi, porta seriale etc.) o eventualmente anche una semplice GUI per questo tipo di configurazione, si può rendere invece l'utilizzo più orientato all'utente. Per la scrittura di "device profile" completi e aderenti allo schema proposto da OpenModbusSpecs (contribuendo così anche a quel progetto) vi sono alcune considerazioni su come gestire metriche non strettamente numeriche (e.g. date) che potrebbero non essere particolarmente utili alla raccolta su Prometheus (per questo è stata aperta una discussione nelle *issues* di OpenModbusSpecs). Utilizzando poi le funzionalità "server" di PyModbus è anche possibile creare un ambiente di sviluppo con test automatizzati e verifica delle funzionalità con pratiche di CI/CD (e.g. GitHub o Forgejo actions) in maniera analoga a quanto fatto manualmente e descritto nel README.md per questo progetto.

Bibliography

[1] Docker. Official documentation, . URL https://docs.docker.com/get-started/docker-overview/.

[2] Docker. Official documentation, . URL https://docs.docker.com/compose/.

[3] Grafana. Official website, . URL https://grafana.com/.

[4] Grafana. Official documentation, . URL https://grafana.com/docs/grafana/latest/.

[5] IBM. Infrastructure monitoring, 2026. URL https://www.ibm.com/it-it/think/topics/infrastructure-monitoring.

[6] O. C. Initiative. Official website. URL https://opencontainers.org/.

[7] OpenModbusSpecs. Github repository. URL https://github.com/stekker/OpenModbusSpecs.

[8] M. Organization. Faq, . URL https://www.modbus.org/faq.

[9] M. Organization. *Modbus application protocol specification*. Schneider Automation Inc., . V1.1b3.

[10] Prometheus. Official documentation, . URL https://prometheus.io/docs/instrumenting/exporters/.

[11] Prometheus. Official documentation, . URL https://prometheus.io/docs/practices/naming/.

[12] Prometheus. Official website, . URL https://prometheus.io/.

[13] Prometheus. Data model, . URL https://prometheus.io/docs/concepts/data_model/.

[14] Prometheus. Official documentation, . URL https://prometheus.io/docs/introduction/overview/.

[15] PyModbus. Github repository. URL https://github.com/pymodbus-dev/pymodbus.

List of Figures

1.1 Semplice dashboard Grafana 3
1.2 Dashboard Grafana con un primo calcolo di PUE 4

2.1 Struttura collector-exporter.py 7